Software Architecture

Version: 1.0

Purpose

This document describes the overall software architecture of Trading Platform Pro.
It defines the architectural principles, system structure and dependency rules that govern the entire project.
Detailed implementation information is documented in the corresponding architecture documents.

Architectural Goals

The architecture is designed to achieve:
• Long-term maintainability
• High modularity
• Strong separation of concerns
• Excellent testability
• Scalability
• Deterministic behavior
• Extensibility
• Enterprise-grade quality

Architectural Style

Trading Platform Pro follows a combination of:
• Clean Architecture
• Domain-Driven Design (DDD)
• Modular Monolith (initially)
• Event-Driven Architecture
• Dependency Injection
This combination provides a solid foundation for future growth while keeping operational complexity low.

High-Level Architecture

```
Presentation
■
▼
Application
■
▼
Domain
▲
■
Infrastructure
```
Dependencies always point toward the Domain layer.

Layer Responsibilities

Presentation

Responsible for:
• User Interface
• Commands
• Views
• View Models
• User Interaction

Application

Responsible for:
• Use Cases
• Commands
• Queries
• Application Services
• Workflow Coordination

Domain

Responsible for:
• Business Rules
• Entities
• Value Objects
• Aggregates
• Domain Services
• Domain Events
The Domain layer contains no infrastructure dependencies.

Infrastructure

Responsible for:
• Persistence
• Configuration
• Logging
• Messaging
• Runtime
• Scheduling
• Plugins
• File System
• External APIs
Infrastructure implements interfaces defined by higher layers.

Dependency Rules

Allowed:
```
Presentation
↓
Application
↓
Domain
Infrastructure
↓
Application
```

↓
Domain
```

Forbidden:
• Domain → Infrastructure
• Domain → Presentation
• Application → Presentation
---
# Cross-Cutting Concerns

Shared infrastructure includes:
• Logging
• Configuration
• Dependency Injection
• Monitoring
• Health Checks
• Serialization
• Runtime Services
These components remain isolated from business logic.
---
# Architectural Principles

Every implementation should strive for:
• High cohesion
• Low coupling
• Explicit dependencies
• Small components
• Testability
• Reusability
• Predictability
---
# Evolution Strategy

The architecture evolves incrementally.
Every new feature shall:
• respect existing boundaries
• preserve dependency direction
• minimize coupling
• maintain consistency
---
# Architectural Decision Records (ADR)

Significant architectural decisions shall be documented as Architecture Decision Records (ADR).
Each ADR should include:
• Context
• Decision
• Alternatives Considered
• Consequences
• Status
Architecture changes must reference the corresponding ADR.
---
# Design Principles

Every architectural decision should promote:
• Separation of Concerns
```

• Single Responsibility
• Explicit Dependencies
• Composition over Inheritance
• Interface Segregation
• Dependency Inversion
Avoid unnecessary complexity.

Scalability

The architecture shall support future expansion without major restructuring.
Future applications may include:
• Portfolio Manager
• Market Scanner
• Strategy Lab
• Reporting Services
All applications shall reuse the shared platform services.

Architecture Review

Before merging architectural changes verify:
• dependency rules preserved
• boundaries respected
• documentation updated
• no cyclic dependencies
• no unnecessary coupling

Related Documents

• Clean_Architecture.md
• Domain_Model.md
• Infrastructure.md
• Project_Structure.md
• AGENTS.md